

# Teaching a Network to Forget: A Walk Through synaptiCITY

*How six words, one weight matrix, and a single line of math explain why Transformers cache everything and brains don't*

synaptiCITY · DataForge 2026: Pathway Track · [github.com/sorathiyalaksh37-lang/synaptiCITY](https://github.com/sorathiyalaksh37-lang/synaptiCITY)

---

## The moment DOG becomes ANIMAL

Open synaptiCITY and the first thing it asks you to do is small: pick DOG as the input, ANIMAL as the output, and press teach. Nothing about this looks like machine learning as you may have pictured it. There is no dataset, no loss curve, no epoch counter. There is a six-word vocabulary — DOG, ANIMAL, PET, CAT, BIRD, FISH — and a 6×6 grid of numbers, all near zero, sitting quietly in memory.

Underneath the button is a single function, `teach()`, and inside it two things happen in order. First, both words are switched on at once: DOG's activation and ANIMAL's activation are each set to 1.0. The code's own comment calls this “fire together.” Second, the connection between them is updated by exactly one formula:

$$\Delta w = \eta \times a_i \times a_j$$

$\eta$  (eta) is the learning rate — in the app it defaults to 0.1.  $a_i$  and  $a_j$  are the activations of the input neuron and the output neuron. Because both DOG and ANIMAL are sitting at 1.0, the update is just  $0.1 \times 1.0 \times 1.0 = 0.1$ , added directly onto the DOG→ANIMAL cell of the weight matrix, then clamped so it never leaves the range [0, 1]. The code's own comment on the update rule is blunt about what it's doing: it “strengthens connections between co-active neurons.” That's the whole mechanism. No backpropagation, no gradient computed against a target, no error signal traveling backward through layers. Two neurons happened to be on simultaneously, and the wire between them got a little thicker.

Press teach a few more times — the app calls this “strengthen it,” stage two of its guided tour — and you watch the same 0.1-scaled nudge land again and again, with the weight climbing toward its ceiling of 1.0 as repetitions accumulate. There's also a quieter side effect: every time you teach DOG→ANIMAL, every connection except that one decays slightly, at a rate of 2% per pulse. The app's `decayOthers()` function loops over the entire matrix and shrinks anything you didn't just reinforce. Memory here isn't just built by use; it's eroded by disuse, in the same call.

## What “plasticity” actually means

“Synaptic plasticity” is a biology term before it's an AI term: it describes the brain's actual capacity for the strength of a connection between two neurons to change based on their joint activity. The classical formulation of this, going back to the neuropsychologist Donald Hebb, is often shortened to “neurons that fire together, wire together.” The equation synaptiCITY runs —  $\Delta w = \eta \times a_i \times a_j$  — is the textbook Hebbian learning rule, not a simplification invented for the app.

What makes this a genuinely different way of storing information, rather than just a smaller neural net, is where the memory lives. A standard Transformer answers a query by comparing the current token against every token it has seen so far, held in a key-value cache that grows with every new token. The weights that define the model are frozen once training ends; nothing about a specific conversation is written into them. The cache is memory, but it's memory bolted onto the outside of the network, scaling linearly with how much text you've fed it. A Hebbian system puts the memory back into the connections themselves. When you teach DOG→ANIMAL in synaptiCITY, the fact that this association exists is now a property of the wiring — one specific cell in a 6×6 matrix — not an entry appended to a growing list somewhere else.

## The fork: teaching DOG a second thing

Stage four of the app's guided tour is called, simply, “the fork.” Having already taught DOG→ANIMAL, you now teach DOG→PET. Same button, same equation, same  $0.1 \times 1.0 \times 1.0$  update — except this time it's landing on a different cell of the matrix: the DOG→PET connection, which starts, like everything else, near zero.

This is where the app hands you its Competing Memory panel, and it's worth describing exactly what you see there, because it's the whole lesson in one screen. The panel draws DOG as a single source node with two branches leaving it: one to ANIMAL, one to PET, and next to each branch it prints the live numeric weight — the actual float sitting in `network.getWeight('DOG','ANIMAL')` and `network.getWeight('DOG','PET')` — to three decimal places. The panel's own copy states it plainly: “You now have two learned paths from DOG [...] Recall selects the strongest learned path.”

Then you hit recall on DOG, and the app's `recall()` function does something specific: it looks at every outgoing connection from DOG and returns the single highest-weighted one as “the” answer, discarding the rest as a ranked list you can still inspect but which the network itself doesn't act on. If you taught ANIMAL more times than PET, DOG still, technically, points to PET — just weakly — but recall only ever surfaces the winner. The app computes a margin between the two weights, and when that margin drops under roughly 0.1 it flags the pair as ambiguous. This is interference made visible: two true, valid associations sharing one input neuron, competing for the same output slot, with the decay

step quietly working against whichever one you reinforce less. Nothing is deleted. Both weights persist in the matrix. But only one of them can win recall at a time, and which one wins depends entirely on your teaching history, not on which fact is more “correct.” That’s the falsifiable claim the whole app is built to let you test: a small Hebbian memory can hold two associations from the same cue, but it cannot hold them at equal, independent strength forever — reinforcing one comes at the expense of the other.

## **The same idea, at a different scale: Dragon Hatchling**

synaptiCITY’s matrix has 36 cells and one learner watching all of them. That’s the entire point — it’s small enough to see. But the Hebbian update rule it runs isn’t a toy invented for teaching; it’s the same mechanism that Pathway’s Dragon Hatchling (BDH) architecture treats as a serious computational proposal for how a Post-Transformer model could work, in Kosowski et al., “The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain” (arXiv:2509.26507).

The paper’s reframing is direct: attention, the mechanism that lets a Transformer decide which earlier tokens matter to the current one, is recast as synaptic memory. Instead of a fixed matrix multiplication run against a stored cache of keys, BDH lets local, synapse-like edge weights get written and updated through Hebbian-style rules as the model reads a sequence — the same “fire together, wire together” logic as the toy model, just running inside a much larger and much sparser network. The reported behavior at that scale is different in kind from what six words can show: roughly 5% of neurons active at any given moment, embedded in a network whose connectivity is scale-free — most units have few connections while a small number become disproportionately connected hubs — with pretraining-scaling experiments the authors report spanning from 1B up to 600B parameters. synaptiCITY makes no attempt to reproduce any of that; it is explicit about being a hand-inspectable illustration of the update rule, not a scaled-down BDH.

## **From association to in-context learning: BDH-CQ**

There’s a second, later step in this line of work worth understanding, because it’s where the Hebbian idea stops being about single word pairs and starts looking like something closer to reasoning. BDH-CQ, described in Engdahl et al., “BDH-CQ: In-Context Learning with Recurrent Latent Reasoning” (arXiv:2608.09888), extends the synaptic-memory mechanism so that demonstrations given to the model at inference time — examples shown in the prompt, not during training — continuously update its recurrent memory. Rather than writing out a step-by-step chain of thought in words, the system works the problem in a high-dimensional latent space through iterative computation, with no parameter updates and, notably, no demonstrations from the evaluation task itself used anywhere in training.

The authors report 29.5% pass@2 on the public ARC-AGI-1 evaluation set, at a computed cost they estimate at roughly \$0.00070 per task. It's worth being precise about what that number is and isn't: it's a figure reported by the system's own developers, not a result independently reproduced by a third party, and it has no connection whatsoever to synaptiCITY — the toy app runs no ARC-AGI evaluation of any kind and has no way to verify it. It's included here only because it's the clearest publicly reported evidence that the Hebbian-style, synaptic-memory idea this whole post walks through in miniature scales up to something that can attempt genuinely hard, abstract reasoning tasks — not because synaptiCITY has anything to say about whether that number holds up under independent scrutiny.

## What the toy app does and doesn't prove

synaptiCITY makes one thing genuinely observable: type two words, watch a specific matrix cell increase by exactly  $\eta \times a_i \times a_j$ , teach a second association from the same cue, and watch the two weights and the recall margin move in front of you in real time, with the interference count and the decay both visibly logged. That's a real, live substrate — the numbers shown are the numbers the network is actually using to answer recall, not an animation standing in for them.

What it can't show is anything about scale. It has no sparsity to speak of — every one of its 36 connections is visible and inspectable at once, where BDH's reported behavior depends on roughly 95% of neurons being inactive at any moment. It has no scale-free, heavy-tailed connectivity; a 6-node graph is too small for a degree distribution to mean anything. And it makes no claim about the monosemantic-synapse behavior BDH reports at scale, where individual connections are said to respond to a specific semantic concept rather than a single fixed token. The most honest way to put it: the toy model shows that Hebbian writes create fast, cheap, real associative memory, and that this memory suffers destructive interference easily when only six words are competing for a handful of connections. Whether that fragility disappears — or simply becomes harder to observe — once the same update rule is embedded in a network with BDH's actual size and sparsity is exactly the question synaptiCITY is built to raise, not to answer.

---

## Sources

- Kosowski, A., Uznanski, P., Chorowski, J., Stamirowska, Z., and Bartoszkiewicz, M. “The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain.” arXiv:2509.26507.
- Engdahl, B., Kosowski, A., Chorowski, J., Stamirowska, Z., Uznanski, P., et al. “BDH-CQ: In-Context Learning with Recurrent Latent Reasoning.” arXiv:2608.09888.
- Code referenced: `src/lib/NeuralNetwork.ts`, `src/App.tsx`, `src/components/CompetingMemoryPanel.tsx` — [github.com/sorathiyalaksh37-lang/synaptiCITY](https://github.com/sorathiyalaksh37-lang/synaptiCITY)